FBServiceExt ხმის მიცემის ბიზნეს ფლოუ

მიმდინარე ოპერატორული ხედვა: **GET_STARTED** ხსნის კანდიდატების არჩევას, შემდეგი დაჭერა ქმნის დადასტურების გამოწვევას, სადაც სწორი ღილაკი რანდომულ პოზიციაზეა, და მხოლოდ სწორი დადასტურება დროში ქმნის მიღებულ ხმას და რთავს cooldown-ს.

რას აჩვენებს ეს დიაგრამა

კანდიდატის არჩევა ჯერ ხმა არ არის. ბიზნეს ეფექტი იქმნება მხოლოდ მაშინ, როცა დადასტურების გამოწვევა წარმატებით სრულდება. ეს არის მიმდინარე ფლოუს მთავარი ანტი-ავტო-კლიკერი.

მიმდინარე RUNTIME მოდელი

კონტენტი, დროები, cooldown, კარუსელის ზარათები და ღილაკების ტექსტებ სრულად config-driven-ია. Messenger შეტყობინებებს worker აგზავნის `MetaMessengerClient`-ით.

- ტრიგერი / გეითი
- არჩევა / სელექცია
- დადასტურება / გამოწვევა
- ხმა მიღებულია / cooldown შედეგი
- config-driven დრო / კონტენტი

ეტაპი 1: ვოტის დაწყება და კანდიდატების კარუსელი

შესვლა ვოტინგის

ნაბიჯი 1

იუზერი აგზავნის GET_STARTED

შემოსული ტექსტი ემთხვევა ერთ-ერთ configured vote-start token-ს.

სწორედ ეს არის ის ტექსტი, რომელიც კანდიდატების კარუსელს ხსნის.

Config ref: `VotingWorkflow:VoteStartTokens` appsettings.json: L54-L58

→

ნაბიჯი 2

worker ამოწმებს token-ს და cooldown-ს

`ProcessMessageAsync()` ignore-ს უკეთებს არა-ბიზნეს ტექსტს და ადრე ჩერდება, თუ იუზერი ჯერ კიდევ cooldown-შია.

თუ cooldown აქტიურია, იუზერი იღებს შემდეგი დროის ტექსტს და კარუსელი არ იგზავნება.

Code ref: NormalizedEventProcessor.cs ProcessMessageAsync(): L183-L274 Cooldown gate: L228-L234

→

ნაბიჯი 3

ინახება `OptionsSent` session

state machine გადადის `Idle → VoteRequested → OptionsSent`.

session ინახავს აქტიურ user scope-ს და options არჩევის expiry-ს.

State refs: L243-L244 Session save + metric: L246-L264

→

ნაბიჯი 4

იგზავნება კანდიდატების კარუსელი

worker აგებს Messenger generic template-ს `Candidates:Items`-დან.

ყველა card იღებს სურათს, display name-ს და ერთ postback ღილაკს.

TrySendCandidateCarouselAsync(): L825-L878 MetaMessengerClient.SendGenericTemplateAsync(): L66-L90

→

შედეგი

Messenger აჩვენებს კარუსელს

თითო enabled კანდიდატზე იქმნება ერთი card, public URL-ებით, რომლებიც იგება configured base URL-ით და asset folder-ით.

ეს არის პირველი ხილული operator-controlled voting surface.

BuildCandidateImageUrl(): L1310 Config refs: appsettings.json L9 L131

ტოკენები

`VotingWorkflow:VoteStartTokens` წყვეტს, რომელი incoming ტექსტი იწყებს voting flow-ს.

კანდიდატები

`Candidates:Items`, `PublicBaseUrl` და `AssetBasePath` განსაზღვრავს კარუსელის კონტენტს და image URL-ებს.

ღილაკის ტექსტი

`MessengerContent:CandidateVoteButtonTitleFormat` მართავს თითო carousel button-ის label-ს.

ეტაპი 2: კანდიდატის არჩევა და დადასტურების გამოწვევა

ანტი-ავტო-კლიკერი

ნაბიჯი 5

იუზერი აჭერს კანდიდატს

ნაბიჯი 6

ნაბიჯი 7

იქმნება `ConfirmationPending`

ნაბიჯი 8

carousel button აგზავნის postback-ს, სადაც `payload = candidate.Id`.

ეს ჯერ კიდევ არ ითვლება ხმად.

Source: carousel button payload

→

resolve-დება active session და candidate

`ProcessPostbackAsync()` ტვირთავს აქტიურ vote session-ს და payload-ით პოულობს კანდიდატს.

თუ active session არ არსებობს ან candidate უცნობია, event ignore-დება.

ProcessPostbackAsync(): L282-L517

→

ფლოუ გადადის `CandidateSelected → ConfirmationPending`.

session უკვე ინახავს არჩეულ კანდიდატს და confirmation timeout window-ს.

Confirmation session build + save: L369-L385

→

იგზავნება ერთ-ბარათიანი დადასტურების გამოწვევა

worker აგზავნის ერთ candidate card-ს prompt ტექსტით და **3 ღილაკით**.

სწორი კანდიდატის label რანდომულად ნაწილდება პოზიციებზე, რომ blind aut click რთული იყოს.

TrySendVoteConfirmationPromptAsync(): L891-L
BuildVoteConfirmationButtons(): L1194-L1235

არასწორი გზა

არასწორი ღილაკი

თუ იუზერი decoy confirmation button-ს დააჭერს, session იშლება და იგზავნება rejected ტექსტი.

Wrong confirmation branch: L423-L449

ძალიან გვიან

ვადაგასული დადასტურება

თუ დრო ამოიწურა, backend reject-ს უკეთებს დადასტურებას და სთხოვს იუზერს თავიდან დაიწყო კანდიდატების არჩევიდან.

HasSessionExpired(): L1306
HandleExpiredSessionAsync(): L767-L822

დრო

`VotingWorkflow:ConfirmationTimeout` ახლა 120 წამია.

პრომტის ტექსტი

`VoteConfirmationPromptFormat` აგებს ტექსტს "დაადასტურეთ არჩევანი N წამში".

ღილაკების რანდომიზაცია

`VoteConfirmationCorrectButtonTitleFormat` და `VoteConfirmationDecoyButtonTitles` კვებავს 3-ღილაკიან ანტი-ავტ კლიკერის გამოწვევას.

ეტაპი 3: ხმა მიღებულია, cooldown და ხელახალი მცდელობა

აქ იქმნება რეალური ბიზნე

ნაბიჯი 9

მოდის სწორი დადასტურება

postback payload ემთხვევა selected candidate-ის expected accept payload-ს.

მხოლოდ ეს branch ქმნის მიღებულ ხმას.

Accept path decision: L403-L417

→

ნაბიჯი 10

`AcceptVoteAsync()`

state machine გადის confirmation-received, persisted, notified და ბოლოს cooldown-active მდგომარეობებზე.

cooldown timestamp სწორედ აქ ითვლება.

AcceptVoteAsync(): L627-L724

→

ნაბიჯი 11

ინახება ხმა და cooldown snapshot

`AcceptedVoteStore.TryAddAsync()` წერს ხმას, ხოლო session store ინახავს cooldown window-ს მომდევნო მცდელობებისთვის.

Accepted vote insert: L667
Cooldown snapshot save: L670-L684

→

ნაბიჯი 12

იგზავნება საბოლოო success ტექსტი

იუზერი ხედავს, რომ ხმა მიღებულია და როდის შეძლებს შემდეგი ხმის მიცემას.

ეს არის წარმატებული vote-ის ხილული დასრულება.

TrySendVoteAcceptedMessageAsync(): L727-L741
BuildVoteAcceptedText(): L1237-L1248

→

შემდეგი მცდელობა

cooldown ბლოკავს ხელახლა ხმას

თუ იუზერი cooldown-ის დ ისევ ცდის, message და pos path ორივე ადრე ჩერდება (აგზავნის შემდეგი დროის ტ

Message cooldown branch: L228-L
Postback cooldown branch: L325-
Cooldown text builder: L1250-L1

cooldown window

`VotingWorkflow:Cooldown` ახლა 5 წუთია და იწყება მხოლოდ სწორი confirmation-ის შემდეგ.

success ტექსტი

`VoteAcceptedTextFormat` აკონტროლებს საბოლოო მესიჯს კანდიდატით და შემდეგი დასაშვები დროით.

retry ტექსტი

`CooldownActiveTextFormat` და `CooldownTimeFormat` აგებს მესიჯს
სცადეთ HH:mm:ss-ის შემდეგ”.

რა ითვლება რეალურ ხმად: carousel-დან კანდიდატის არჩევა მხოლოდ selection-ია. ბიზნეს vote იქმნება მხოლოდ მაშინ, როცა confirmation challenge დროში წარმატებით დასრულდება.

countdown-ის წიაანსი: Messenger generic/button template-ს შეუძლია აჩვენოს configured `120 წამ`
როგორც static ტექსტი, მაგრამ native live ticking countdown bubble არ აქვს. რეალური enforce
მაინც backend session expiration-სა და cooldown logic-ში ხდება.